PyPTO: 模型融合算子实操

作者：王子楠——PyPTO 仓 Committer

时间：2026/03/12

课程背景与目标

课程背景：

- 作为 PyPTO 系列课程的第四讲，本节课将在前三讲的基础上，系统介绍如何利用 PyPTO 在昇腾 NPU 上进行大模型的融合算子开发，完整阐述其开发全流程。

课程目标：

- 明确使用 PyPTO 进行融合算子开发的流程和必需的准备工作
- 掌握模块化地编写 PyPTO 融合算子的要点
- 具备基础的融合算子优化能力，学习进阶优化的技巧
- 了解下一代硬件对 PyPTO 算子开发的影响

前置知识：

- PyPTO 的基础概念
- 熟悉 PyTorch 算子开发

CANN

目录

Part 1 从计算流出发

Part 2 融合算子的开发

Part 3 再谈性能调优

Part 4 下一代硬件的编程

模型融合算子开发流程图

开发流程图

阅读模型脚本，进行融合算子开发准备

- 梳理计算流，分析算子开发需求
- 绘制计算流程图
- 明确算子原型，确定输入输出

根据计算流完成对应模块的开发与组装，完成精度调试

利用 PyPTO Toolkit 工具，分析性能优化点，从基础到进阶完成性能优化

CANN

从计算流出发：分析开发需求

阅读模型代码，梳理计算流

- 右图所示是 DeepSeek V3.2 Indexer 模块部分片段。我们对这一段代码进行适配，写成一个融合算子，取名为 IndexerProlog

- 为什么要写成融合算子?
  - 减少 kernel launch
  - 减少不必要的数据搬运
  - 提升计算并行度

- 梳理计算流和开发需求
  - Cube：linear & rotate_activation
  - Vec：RoPE & act_quant & norm

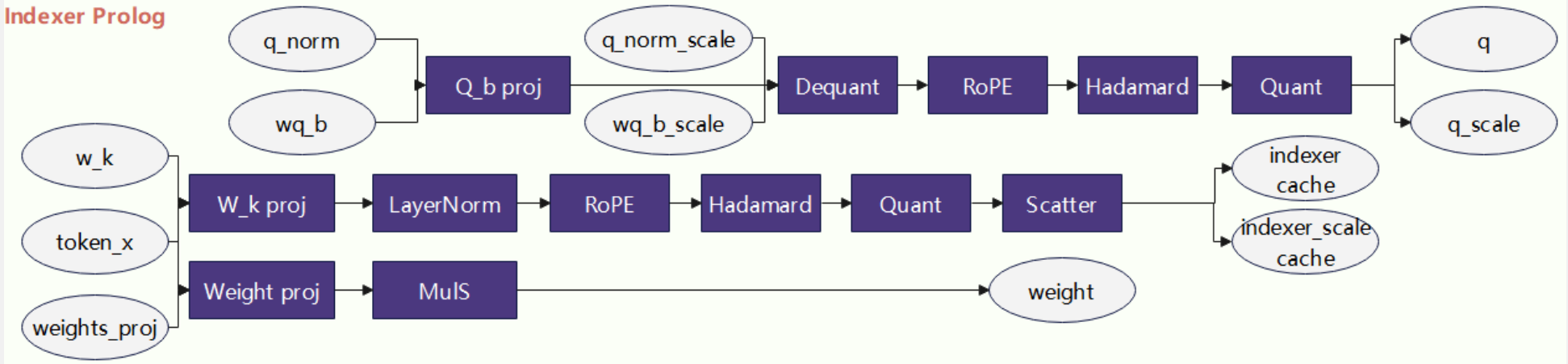
```
def forward(self, x: torch.Tensor, qr: torch.Tensor, start_pos: int, freqs_cis: torch.Tensor,
mask: Optional[torch.Tensor]):
bsz, seqlen, _ = x.size()
end_pos = start_pos + seqlen
q = self.wq_b(qr)
q = q.view(bsz, seqlen, self.n_heads, self.head_dim)
q_pe, q_nope = torch.split(q, [self.rope_head_dim, self.head_dim - self.rope_head_dim],
dim=-1)
# rope in indexer is not interleaved
q_pe = apply_rotary_emb(q_pe, freqs_cis, False)
q = torch.cat([q_pe, q_nope], dim=-1)
k = self.wk(x)
k = self.k_norm(k)
k_pe, k_nope = torch.split(k, [self.rope_head_dim, self.head_dim - self.rope_head_dim],
dim=-1)
# rope in indexer is not interleaved
k_pe = apply_rotary_emb(k_pe.unsqueeze(2), freqs_cis, False).squeeze(2)
k = torch.cat([k_pe, k_nope], dim=-1)
q = rotate_activation(q)
k = rotate_activation(k)
q_fp8, q_scale = act_quant(q, block_size, self.scale_fmt)
k_fp8, k_scale = act_quant(k, block_size, self.scale_fmt)
self.k_cache[:bsz, start_pos:end_pos] = k_fp8
self.k_scale_cache[:bsz, start_pos:end_pos] = k_scale
weights = self.weights_proj(x.float()) * self.n_heads ** -0.5
weights = weights.unsqueeze(-1) * q_scale * self.softmax_scale
```

CANN

从计算流出发：绘制计算图

根据计算流，画出计算流程图

- 梳理完计算流后，绘制计算流程图



CANN

从计算流出发：制定算子原型

根据计算流程图，制定算子原型

- 根据计算流程图，制定算子原型表格

字段分组	字段名称	字段类型	字段类型	shape	Format	备注
INPUT 0	q_norm	tensor	DT_INT8	(t, q_lora_rank)	ND	
INPUT 1	q_norm_scale	tensor	DT_FP32	(t, 1)	ND	
INPUT 2	token_x	tensor	DT_BF16	(t, h)	ND	
INPUT 3	w_qb	tensor	DT_INT8	(idx_n_heads*idx_head_dim//32, q_lora_rank//16, 16, 32)	NZ	实际 shape: (q_lora_rank, idx_n_heads*idx_head_dim)
INPUT 4	w_qb_scale	tensor	DT_FP32	(idx_n_heads*idx_head_dim, 1)	ND	Linear 反量化因子，-1 轴维度 =1
INPUT 5	w_k	tensor	DT_BF16	(idx_head_dim//16, h//16, 16, 16)	NZ	实际 shape: (h, idx_head_dim)
INPUT 6	w_proj	tensor	DT_BF16	(idx_n_heads//16, h//16, 16, 16)	NZ	实际 shape: (h, idx_n_heads)
INPUT 7	layernorm_gamma_k	tensor	DT_BF16	(idx_head_dim,)	ND	
INPUT 8	layernorm_beta_k	tensor	DT_BF16	(idx_head_dim,)	ND	
INPUT 9	rope_cos	tensor	DT_BF16	(t, rope_dim)	ND	
INPUT 10	rope_sin	tensor	DT_BF16	(t, rope_dim)	ND	
INPUT 11	hadamard_q	tensor	DT_BF16	(idx_head_dim, idx_head_dim)	ND	
INPUT 12	hadamard_k	tensor	DT_BF16	(idx_head_dim, idx_head_dim)	ND	
INPUT 13	idx_k_cache	tensor	DT_INT8	(block_num, block_size, n_kv, idx_head_dim)	PA_BSND	输入输出复用
INPUT 14	idx_k_scale_cache	tensor	DT_FP16	(idx_block_num, block_size, n_kv)	PA_BSND	输入输出复用
INPUT 15	cache_index	tensor	DT_INT64	(t,)	ND	
ATTR 0	layernorm_epsilon_k	FLOAT	\	\	\	
ATTR 1	score_scale	FLOAT	\	\	\	
ATTR 2	layout_key	STRING	\	\	\	
OUTPUT 0	idx_q	tensor	DT_INT8	(t, idx_n_heads, idx_head_dim)	ND	
OUTPUT 1	idx_q_scale	tensor	DT_FP16	(t, idx_n_heads)	ND	
OUTPUT 2	weights	tensor	DT_FP16	(t, idx_n_heads)	ND	

CANN

本章小结

- **本章介绍了如何从网络模型脚本，开发新的融合算子的准备工作**
  - PyPTO 开发融合算子以计算流为基础
  - 从计算流出发，绘制计算流程图，以便后续开发
  - 制定算子原型，明确输入、属性和输出

CANN

目录

Part 1 从计算流出发

Part 2 融合算子的开发

Part 3 再谈性能调优

Part 4 下一代硬件的编程

PyPTO 融合算子开发的特点

PyPTO 开发特点
• 从计算流出发，了解数学公式即可着手开发
• 无需感知硬件细节，专注数学逻辑的表达
• 框架进行自动融合，减少 kernel launch 和不必要的数据搬运
• 优先确保精度和功能上的正确，精度通过 PyTorch 编写 golden 进行比对

CANN

融合算子开发：按模块进行开发

Layernorm

- 根据 Layernorm 的计算流，编写 Layernorm

```
def layer_norm(x: pypto.Tensor, gamma: pypto.Tensor, beta: pypto.Tensor, dim: int, epsilon: float):
    assert ((dim == len(x.shape) - 1) or (dim == -1))
    actual_dim = dim + len(x.shape) if dim < 0 else dim
    x_dtype = x.dtype

    # numerator
    x_fp32 = pypto.cast(x, pypto.DT_FP32)
    # do division first to avoid overflow
    x_scaled = x_fp32 * (1.0 / x.shape[actual_dim])
    mean = pypto.sum(x_scaled, actual_dim, keepdim=True)
    diff = x_fp32 - mean

    # denominator
    squared_diff = diff * diff
    squared_diff_scaled = squared_diff * (1.0 / x.shape[actual_dim])
    var = pypto.sum(squared_diff_scaled, actual_dim, keepdim=True)
    # add epsilon to avoid division by zero
    var_eps = var + epsilon
    std_var = pypto.sqrt(var_eps)
    res32 = diff / std_var

    # result
    gamma32 = pypto.cast(gamma, pypto.DT_FP32)
    beta32 = pypto.cast(beta, pypto.DT_FP32)
    return pypto.cast((res32 * gamma32) + beta32, x_dtype)
```

$$y = \frac{x - E(x)}{\sqrt{Var(x) + \epsilon}} * \gamma + \beta$$

CANN

融合算子开发：按模块进行开发

Per-token-dynamic-quant

- 这里采用了细粒度的（Per-token）动态量化

```
def quant(x: pypto.Tensor):
    s8_max_value = 127.0
    s8_one_value = 1.0
    input_fp32 = pypto.cast(x, pypto.DT_FP32, pypto.CastMode.CAST_NONE)

    abs_res = pypto.abs(input_fp32)
    max_value = pypto.amax(abs_res, dim=-1, keepdim=True)
    temp127 = pypto.full(max_value.shape, s8_max_value, pypto.DT_FP32)

    scale_quant = temp127 / max_value
    out_fp32 = input_fp32 * scale_quant
    out_int32 = pypto.cast(out_fp32, pypto.DT_INT32, pypto.CastMode.CAST_RINT)
    out_half = pypto.cast(out_int32, pypto.DT_FP16, pypto.CastMode.CAST_ROUND)
    out_int8 = pypto.cast(out_half, pypto.DT_INT8, pypto.CastMode.CAST_TRUNC)
    temp1 = pypto.full(scale_quant.shape, s8_one_value, pypto.DT_FP32)
    scale_dequant = temp1 / scale_quant
    return (out_int8, scale_dequant)
```

融合算子开发：按模块进行开发

Non-interleaved RoPE

- 根据 Non-interleaved RoPE 模块，编写 RoPE
- 分为 rotate_half 和 rope 部分

```
def rotate_half(input_tensor: pypto.Tensor) -> pypto.Tensor:
    chunk_size = 2
    shape = input_tensor.shape
    shape_size = len(shape)
    assert shape_size >= 1
    assert shape[shape_size - 1] % chunk_size == 0
    shape[shape_size - 1] //= chunk_size
    offset1 = [0] * shape_size
    offset2 = [0] * shape_size
    offset2[shape_size - 1] = shape[shape_size - 1]
    x1 = pypto.view(input_tensor, shape, offset1)
    x2 = pypto.view(input_tensor, shape, offset2)
    return pypto.concat([x2 * (-1.0), x1 + 0.0], -1)
```

```
def rope_3d(x: pypto.Tensor, cos: pypto.Tensor, sin: pypto.Tensor, configs: IndexerPrologQuantConfigs) -> pypto.Tensor:
    head_num_axis = 1
    head_dim_axis = 2
    assert (len(x.shape) == SHAPE_DIM_3 and len(cos.shape) == SHAPE_DIM_2 and len(sin.shape) == SHAPE_DIM_2)
    x_dtype = x.dtype
    t_tile = x.shape[0]
    head_num = x.shape[head_num_axis]
    rope_dim = x.shape[head_dim_axis]
    pypto.set_vec_tile_shapes(512, rope_dim)
    cast_cos = pypto.cast(cos, pypto.DT_FP32)
    cast_sin = pypto.cast(sin, pypto.DT_FP32)
    pypto.set_vec_tile_shapes(16, head_num, rope_dim)
    x_view = pypto.cast(x, pypto.DT_FP32)
    cast_cos = pypto.reshape(cast_cos, [t_tile, 1, rope_dim])
    cast_sin = pypto.reshape(cast_sin, [t_tile, 1, rope_dim])
    x_embed = (x_view * cast_cos) + ((rotate_half(x_view)) * cast_sin)
    res = pypto.cast(x_embed, x_dtype)
    return res
```

CANN

本章小结

- **本章阐述了 PyPTO 融合算子开发的方式**
  - 根据梳理出的计算流，先完成对应的模块，例如 Layernorm/RoPE/Quant 的开发
  - 之后再将这些模块组装起来，完成融合算子开发

CANN

目录

Part 1 从计算流出发

Part 2 融合算子的开发

Part 3 再谈性能优化

Part 4 下一代硬件的编程

调优流程

PyPTO 性能调优基本步骤

- 按照对性能影响大小的大致排序
  - 动态的分档与 stitch 功能(静态图合并)
  - Cube & Vector 的性能(由泳道图分析当前的瓶颈)
  - 拓扑关系(依赖关系是否复杂)
  - 其他调度策略
- 注意事项
  - 优先调整对性能影响大的部分，调整好后，再逐步调整其他模块；一般来说，优先调整动态轴的分档，提高静态图的合并能力

以 IndexerProlog 算子 Prefill 场景算子调优为例，我们优先调整静态图合并力度：

- 相比于 Decode 场景，Prefill 场景的最大区别是动态轴 t（=batch_size * seq_len）显著增大。针对这种动态轴范围较广的时候，可以采用 loop_unroll 的写法；
- 在 Decode 场景下，我们 unroll_list 为：[32, 16, 8, 4, 2, 1] 即可满足使用，而在 Prefill 场景下，可以改为：[512, 128, 32, 8, 2, 1]
- 通过增大 unroll_list 最大值，我们可以在 Prefill场景下，显著提高静态图的合并力度，从而提高性能

Cube/Vec 性能调优基础

Cube/Vector 的性能调优的基本原则

- Cube 性能调优
  - 在 Decode 场景下，通常是带宽 bound，可以通过搬运量对理论性能进行估算
  - Prefill 和 Decode 场景的最优 tile 切分通常有所不同，可以利用 loop_unroll 来解决
- Vector 性能调优
  - 调整 TileShape 是 vector 优化的最主要工作
  - 涉及到归约操作，尽量不切归约轴
  - 尾轴的 TileShape 尽量切大，尽量满足为 256B 的倍数，亲和搬运和计算
  - 连续的 Vector 操作尽量使用相同的 TileShape

- Cube 性能调优例子
  - Prefill 场景下：Q-linear 矩阵乘的 TileShape 为 [t_tile_max, t_tile_max], [512, 512], [128, 128] 性能会得到提升，其中 t_tile_max = unroll_list[1]，可以选取 unrollist 其中的一个较为合适的值
  - K-linear、weights-linear 矩阵乘 TileShape 可与 Q-linear 不一样，追求最优性能
- Vec 性能调优例子
  - Layernorm 的尾轴(归约轴不切)
  - 诸如 Layernorm/Quant 包含了连续的 Vector 操作，使用相同的 TileShape，从而保证子图的合并

CANN

Cube/Vec 更进阶的性能调优技巧

Cube/Vector 的进阶调优

- Cube 性能进阶
  - 对于左/右矩阵存在重复利用的时候，可以采用 L1Reuse/CubeNBuffer 进行子图合并，减少冗余搬运以及任务调度的开销
- Vector 性能进阶
  - TileShape 不宜设置过大，容易导致 OoO spill 问题，导致冗余的 UB->GM->UB 的搬运
  - TileShape 较小时，可以通过 VecNBuffer 将同构的子图进行合并，提升效率

- Cube 性能进阶例子
  - Hadamard Transform 的矩阵乘中，右矩阵是固定的 [128, 128] 的大小，此时可以通过 L1Reuse 的功能，避免右矩阵的重复搬运，同时减少任务数，从而提升调度开销
- Vector 性能进阶例子
  - Q RoPE 的计算中，避免 TileShape 过大，尽管下一代硬件的 UB 增大，但是仍然有限制，当单个 Tile 块达到 96KB(FP32 数据类型）时，会出现 OoO spill，即框架会将一些暂时占据空间的 buffer 进行搬出，等部分计算释放空间后再搬入，但是冗余的搬运影响性能，可通过将单个 Tile 块缩小到 64KB 甚至更小去避免该问题

利用 AI Agent 实现自主调优

AI Agent 实现算子开发与调优

- 使能 Agent 调优正在开发中

2026Q1

阶段1：
小型融合算子生成&社区发布

小型融合算子自动生成：
- 资料和案例的整理和补充
- 算子开发工作流Skill上线

算力平台 CANNBot 发布：
- Skill上社区，生态共建
- 迭代 CANNBot 使用指南

2026Q2

阶段2：
中等融合算子生成&性能调优

中等融合算子自动生成：
- 基于精度工具的Skill上线
- 支持定位和修复算子功能、精度问题

支持性能迭代调优：
- 性能自动调优进阶Skill上线
- 支持自动迭代调优性能

主流生态兼容性：
- 常见算子自动迁移

2026H2

阶段3：
复杂融合算子&深度调优

复杂融合算子自动生成：
- 基于模型自主分析和规划算子

深度性能调优：
- 基于泳道图等自动识别优化点
- 支持aicpu耗时优化

Agent自我迭代：
- 扩展“评估agent“能力，针对Skill、资料等支持在线优化

https://gitcode.com/cann

CANN

本章小结

- **本章对 PyPTO 的调优进行了更深层次的探讨**
  - 介绍了基本的调优思路，调优从影响性能最大要素开始
  - 通过 PyPTO Toolkit 工具先调优性能瓶颈点，避免在全局差的情况进行局部调优
  - Cube 和 Vector 的算子的 TileShape 设置有一些基本的原则，需要根据场景灵活调整 TileShape
  - 在 TileShape 调整后，可以通过 L1Reuse/CubeNBuffer/VecNBuffer 进行更进一步地细致调优
  - AI agent 自主调优的功能正在开发中

CANN

目录

Part 1 从计算流出发

Part 2 融合算子的开发

Part 3 再谈性能调优

Part 4 下一代硬件的编程

下一代硬件特性：FP8/HiFloat8 per token 量化的支持

Per-token-dynamic-quant 对 FP8/HiFloat8 的使能

```
def fp8e4m3_quant(x: pypto.Tensor):
    fp8_max_value = 448.0
    fp8_one_value = 1.0
    input_fp32 = pypto.cast(x, pypto.DT_FP32,
pypto.CastMode.CAST_NONE)

    abs_res = pypto.abs(input_fp32)
    max_value = pypto.amax(abs_res, dim=-1, keepdim=True)
    temp448 = pypto.full(max_value.shape, fp8_max_value,
pypto.DT_FP32)

    scale_quant = temp448 / max_value
    out_fp32 = input_fp32 * scale_quant
    out_fp8 = pypto.cast(out_fp32, pypto.DT_FP8E4M3)
    temp1 = pypto.full(scale_quant.shape, fp8_one_value,
pypto.DT_FP32)
    scale_dequant = temp1 / scale_quant
    return (out_fp8, scale_dequant, out_fp32)
```

```
def hifloat8_quant(x: pypto.Tensor):
    hif8_max_value = 32768.0
    hif8_one_value = 1.0
    input_fp32 = pypto.cast(x, pypto.DT_FP32,
pypto.CastMode.CAST_NONE)

    abs_res = pypto.abs(input_fp32)
    max_value = pypto.amax(abs_res, dim=-1, keepdim=True)
    temp32768 = pypto.full(max_value.shape, hif8_max_value,
pypto.DT_FP32)

    scale_quant = temp32768 / max_value
    out_fp32 = input_fp32 * scale_quant
    out_hif8 = pypto.cast(out_fp32, pypto.DT_HF8)
    temp1 = pypto.full(scale_quant.shape, hif8_one_value,
pypto.DT_FP32)
    scale_dequant = temp1 / scale_quant
    return (out_hif8, scale_dequant)
```

CANN

下一代硬件特性：MXFP8 矩阵乘法

MXFP8 的矩阵乘法

- 下一代硬件支持了 MXFP8 数据类型的矩阵乘法，PyPTO 进行了适配
- 接口定义如下：scaled_mm(mat_a, mat_b, out_dtype, scale_a, scale_b, *, a_trans = False, b_trans = False, scale_a_trans = False, scale_b_trans = False, c_matrix_nz = False, extend_params=None) -> Tensor
- 详细的接口使用可以参考 API：https://gitcode.com/cann/pypto/blob/master/docs/api/operation/pypto-scaled_mm.md

下一代硬件特性：通路的使能——mix 子图切分

mix 子图切分

- 下一代硬件增加了 UB->L1，L0C->UB 的数据通路，为了使能这一通路，PyPTO 在 pass 组件提供了 mix 子图功能

- mix 就是把相邻的aic子图和aiv子图合并成一个 mix 子图，以使能一组 core(1个aic和2个aiv)之间的内部数据通路，而不是让数据通过DDR来转移。优点是更高效的数据传输。涉及的pass主要是ReduceCopy，负责将CV分离的子图合并成MIX子图；以及OoO_Schedule中的CoreSchedule，负责将一个mix子图中的op合理分配到1C2V的核心上；以及MixSubgraphSplit，负责将MIX子图差分成单独的AIC和AIV以让machine执行。

https://gitcode.com/cann

CANN

下一代硬件特性：VF 融合

VF 融合

- 下一代硬件提供了 VF 融合的能力
- PyPTO 通过 Tile Fusion，即 PTO BiSheng 编译器为 PTO Tile 编程模型专门设计的优化 Pass。其核心目的是极致使能下一代芯片 VF (Vector Fusion) 融合能力，从而提升 Vector Core 的性能。
- 可融合的指令列表如下：

类别	具体支持指令
Element-wise	`TAdd`, `TSub`, `TMul`, `TDiv`, `TAddS`, `TSubS`, `TDivS`, `TMulS`, `TMax`, `TMin`, `TMinS`, `TExp`, `TSqrt`, `TRSqrt`, `TRecip`, `TCvt`, `TMov`
Expand	`TRowExpand`, `TRowExpandAdd`, `TRowExpandSUB`, `TRowExpandMUL`, `TRowExpandDiv`, `TRowExpandMax`, `TRowExpandMin`, `TRowExpandExpDif`
Reduce	`TRowMax`, `TRowMin`, `TRowSum`

https://gitcode.com/cann

CANN

本章小结

- **本章阐述了面对下一代硬件平台的新特性，PyPTO 进行的适配工作**
  - 新的数据类型：FP8/HiFloat 相关的量化算法实现
  - MXFP8 的矩阵乘法：scaled_mm 接口
  - 数据通路：通过 pass 的 mix 子图实现
  - VF 融合能力：通过 Bisheng 编译器优化 pass 实现
  - 面对下一代硬件平台，通过 Tensor 层次编程，屏蔽硬件细节的感知，算子开发者以较低的学习成本和迁移成本，高效地完成开发

Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.

上CANN社区获取干货

关注CANN公众号获取资讯

https://gitcode.com/cann